Universidad Galileo

Facultad de Ingeniería de Sistemas, Informática y Ciencias de la Computación (FISICC)

Curso: Cloud Computing

Catedrático: Adrián Catalán

AquaControl 360

Plataforma de gestión acuícola desplegada en Google Cloud

Documentación del proyecto final: arquitectura, servicios, despliegue, costos y seguridad

Integrante
Allan Gómez
Andrés Culajay

Guatemala, __10__ de noviembre de 2026 (versión corregida)

Contenido

1. Enlaces de la entrega y cambios
2. Descripción del proyecto
3. Arquitectura en la nube
4. Servicios de Google Cloud utilizados
5. Flujos principales entre servicios
6. Decisiones de arquitectura
7. Instrucciones de despliegue paso a paso
8. Consideraciones de seguridad
9. Estimación de costos
10. Escalabilidad y manejo de recursos
11. Limitaciones y trabajo futuro
12. Cumplimiento de la rúbrica

Referencias

1. Enlaces de la entrega

Entregable	Enlace
Sitio web desplegado (Cloud Run, aquacontrol-web)	https://aquacontrol-web-1000315935309.us-central1.run.app
API REST desplegada (Cloud Run, aquacontrol-api)	https://aquacontrol-api-1000315935309.us-central1.run.app
Video de demostración (5 a 7 min)	https://drive.google.com/drive/folders/1MWxFxVh4hRcIK-OCu-Yn1zowcs6nbxCG?usp=sharing
Repositorio de código (público)	https://github.com/VegettoSsjBlue/ERP-para-Acuicultura (rama BD-actualizada)
Credenciales de prueba	Se enviaron al catedrático por mensaje privado; no se incluyen en este documento.

La API responde únicamente a solicitudes autenticadas, salvo el inicio de sesión. Para probarla se usa el sitio web o las aplicaciones móviles con las credenciales de prueba enviadas por mensaje privado.

1.1 Cambios respecto a la entrega del 8 de noviembre

Esta versión atiende las observaciones de la revisión:

- **Formato:** la documentación se entrega en PDF.
- **Video:** se agregó el video de demostración (enlace en la tabla anterior).
- **Repositorio:** ahora es público; antes de abrirlo se revisó que el historial no contenga secretos.
- **Evidencia:** la sección 4.1 incluye capturas de la consola de cada uno de los 10 servicios, y el paso 12 de la sección 7 trae los comandos para verificarlos.
- **Pub/Sub:** se eliminó el token de la URL de la suscripción. Ahora cada mensaje llega firmado con un token OIDC de la cuenta de servicio pubsub-push, y la API verifica la firma antes de procesarlo (secciones 5.1, 7 y 8).
- **Credenciales:** el documento ya no publica usuarios ni contraseñas, y las contraseñas de los usuarios de prueba se cambiaron.

2. Descripción del proyecto

AquaControl 360 es una plataforma de gestión para granjas acuícolas de camarón y tilapia, con un prototipo adicional de ganadería. Nació como una aplicación que

monitoreaba el oxígeno de las piscinas y encendía los aireadores, y en este semestre se transformó en un sistema completo con un backend propio, una base de datos relacional y tres tipos de cliente: el sitio web, las aplicaciones móviles (Android y una versión iOS) y los nodos IoT con ESP32 instalados en cada piscina.

El problema que resuelve es concreto: si el oxígeno disuelto baja durante la noche, el camarón puede morir en pocas horas. Por eso el sistema mide continuamente, avisa al personal y permite encender los aireadores desde el teléfono. Además lleva el control administrativo de la granja: ciclos de producción, muestreos con atarrayazos, gastos, cosechas, ventas, trabajadores y sus pagos.

Funciones que están en producción en la nube:

- Inicio de sesión con JWT y tres roles (Dueño, Administrador y Trabajador), cada uno con su propia vista.
- Monitoreo de piscinas con lecturas reales del ESP32 (oxígeno disuelto y temperatura) y lecturas simuladas por software en el segundo nodo.
- Control remoto de actuadores (aireador y luces) desde la web y el celular; el ESP32 consulta su estado cada 3 segundos.
- Alertas de oxígeno bajo (menor a 4 mg/L) generadas a partir de las lecturas que pasan por Pub/Sub.
- Ciclos de producción, muestreos y atarrayazos, gastos, trabajadores y pagos, clientes, cosechas y ventas.
- Asistente de IA con Gemini (Vertex AI) que responde preguntas sobre la granja.
- Prototipo de ganadería: una ESP32-CAM envía fotos del potrero, se guardan en Cloud Storage y Gemini cuenta los animales y devuelve un recuadro por cada uno.

Firebase no forma parte del sistema. La versión anterior lo usaba para datos y autenticación; todo se migró a Cloud SQL y a un sistema de usuarios propio con contraseñas cifradas con BCrypt, por lo que ningún servicio de Firebase se cuenta dentro de los servicios de la nube.

3. Arquitectura en la nube

Todo el sistema gira alrededor de un solo backend en Cloud Run. Los clientes nunca se conectan directamente a la base de datos ni al bucket: hablan con la API por HTTPS y es la API la única que tiene permisos sobre los datos. La figura 1 muestra la arquitectura tal como está desplegada.

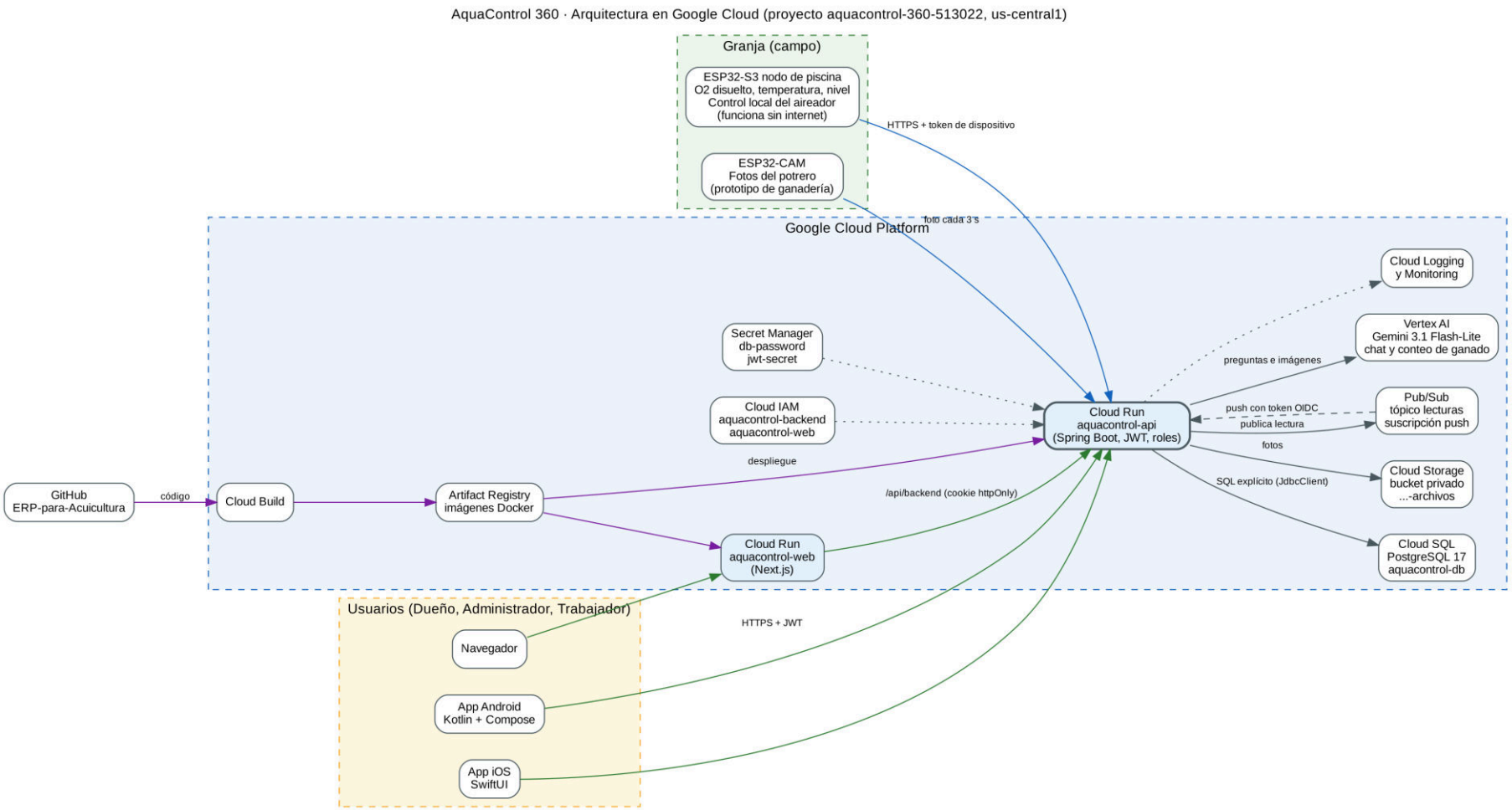


Figura 1. Arquitectura de AquaControl 360 en Google Cloud (proyecto aquacontrol-360-513022, región us-central1). Elaboración propia.

Los componentes se organizan en cuatro capas:

- **Campo:** el nodo ESP32-S3 de cada piscina lee los sensores, decide localmente si enciende el aireador cuando el oxígeno baja (aunque no haya internet) y envía las lecturas a la API. La ESP32-CAM envía fotos del potrero para el prototipo de ganadería.
- **Clientes:** el sitio web en Next.js, que también corre en Cloud Run, y las aplicaciones Android e iOS. Todos consumen la misma API REST.
- **Servicios de aplicación y datos:** la API en Spring Boot (Cloud Run), Cloud SQL con PostgreSQL, Cloud Storage para archivos, Pub/Sub para las lecturas y Vertex AI para la inteligencia artificial.
- **Seguridad, operación y despliegue:** IAM, Secret Manager, Cloud Logging y Monitoring, Cloud Build y Artifact Registry.

4. Servicios de Google Cloud utilizados

El proyecto usa **10 servicios** de Google Cloud configurados directamente desde la consola y la CLI de gcloud. La tabla 1 resume el papel de cada uno y la configuración real.

#	Servicio	Categoría	Rol en AquaControl	Configuración
1	Cloud Run	Cómputo	Ejecuta la API (aquacontrol-api) y el sitio web (aquacontrol-web) en contenedores, con HTTPS y escalado automático.	us-central1, 512 MiB, mínimo 0 y máximo 2 instancias en la API.
2	Cloud SQL (PostgreSQL)	Base de datos	Guarda todos los datos del negocio: usuarios, roles, granjas, piscinas, ciclos, muestreos, lecturas, alertas, gastos, ventas y ganadería (36 tablas normalizadas a 3FN).	Instancia aquacontrol-db, PostgreSQL 17, db-f1-micro, 10 GB SSD, zonal, respaldo diario 08:00 UTC.
3	Cloud Storage	Almacenamiento	Guarda las fotos de la ESP32-CAM y, a futuro, imágenes de muestreos y comprobantes. En PostgreSQL solo se guarda la ruta.	Bucket aquacontrol-360-513022-archivos, acceso uniforme y prevención de acceso público.
4	Pub/Sub	Mensajería	Recibe cada lectura del ESP32 y la entrega a la API por suscripción push; si el procesamiento falla, reintenta y la lectura no se pierde.	Tópico lecturas, suscripción push lecturas-a-procesador con autenticación OIDC, reintentos de 10 s a 10 min.
5	Vertex AI (Gemini)	Inteligencia artificial	Chat de consultas de la granja y conteo de ganado en imágenes con recuadros por animal.	Modelo gemini-3.1-flash-lite, ubicación global, rol aiplatform.user solo para el backend.
6	Secret Manager	Seguridad	Guarda la contraseña de la base y la clave de firma de los JWT. Nada de esto está en el código ni en el repositorio.	Secretos db-password y jwt-secret, montados como variables en Cloud Run.

7	Cloud IAM	Seguridad	Cuentas de servicio propias con permisos mínimos y roles limitados para cada integrante.	aquacontrol-backend (Cloud SQL Client, Secret Accessor, Storage Object Admin en el bucket, AI Platform User, Pub/Sub Publisher en el tópico), aquacontrol-web sin permisos y pubsub-push, que solo firma los mensajes de la suscripción.
8	Cloud Build	CI/CD	Compila el backend y la web a partir del código fuente y genera las imágenes Docker.	Se ejecuta con gcloud builds submit y gcloud run deploy --source.
9	Artifact Registry	CI/CD	Almacena las imágenes Docker versionadas (backend:v1, v2, v3...) de las que arranca Cloud Run.	Repositorio Docker aquacontrol en us-central1.
10	Cloud Logging y Monitoring	Observabilidad	Registra los logs de Spring Boot y de Next.js, y muestra métricas de solicitudes, latencia, errores e instancias de Cloud Run y Cloud SQL.	Integración automática de Cloud Run; consulta desde la pestaña Registros y Métricas del servicio.

Tabla 1. Servicios utilizados y su configuración.

4.1 Evidencia en la consola de Google Cloud

Las siguientes capturas muestran cada servicio creado y en funcionamiento dentro del proyecto aquacontrol-360-513022.

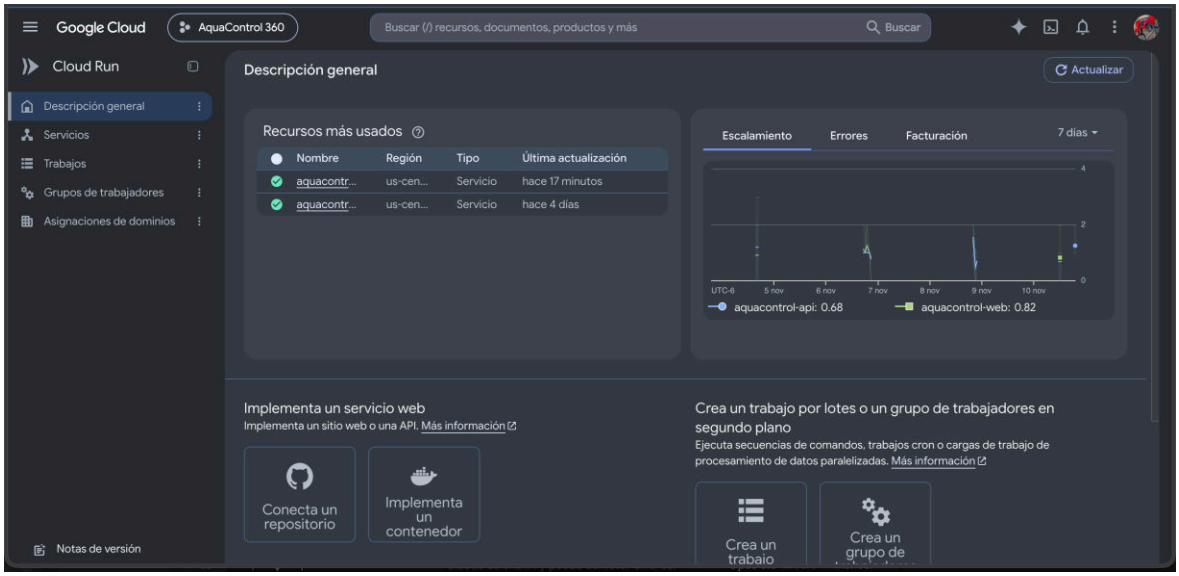


Figura 2. Cloud Run: servicios aquacontrol-api y aquacontrol-web activos, con su escalamiento de los últimos 7 días. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

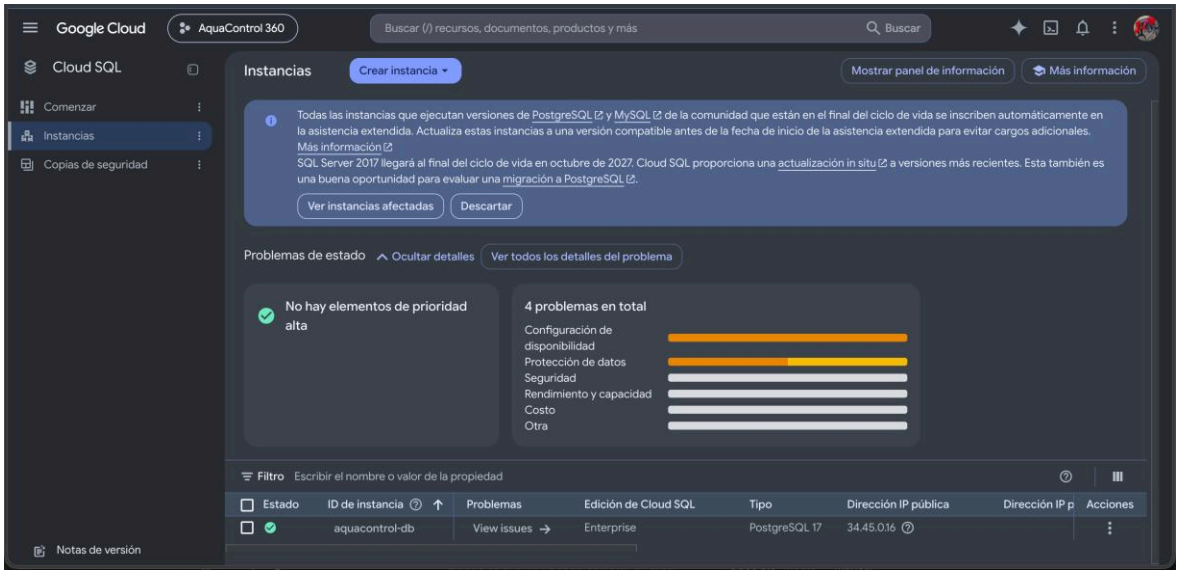


Figura 3. Cloud SQL: instancia aquacontrol-db, edición Enterprise, PostgreSQL 17, en ejecución. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

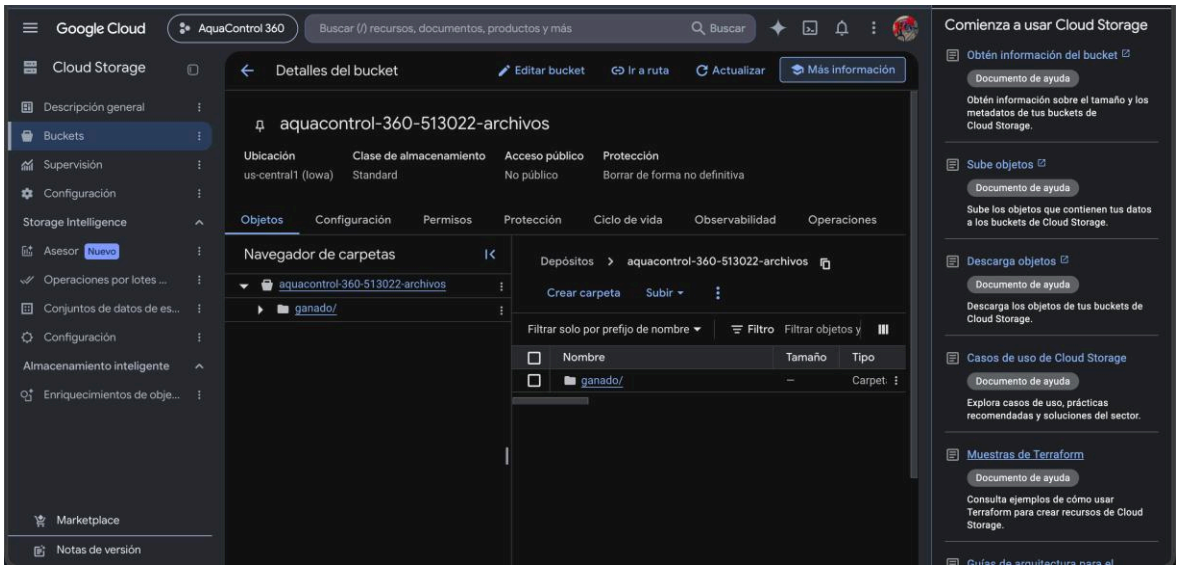


Figura 4. Cloud Storage: bucket aquacontrol-360-513022-archivos en us-central1, no público, con la carpeta ganado/. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

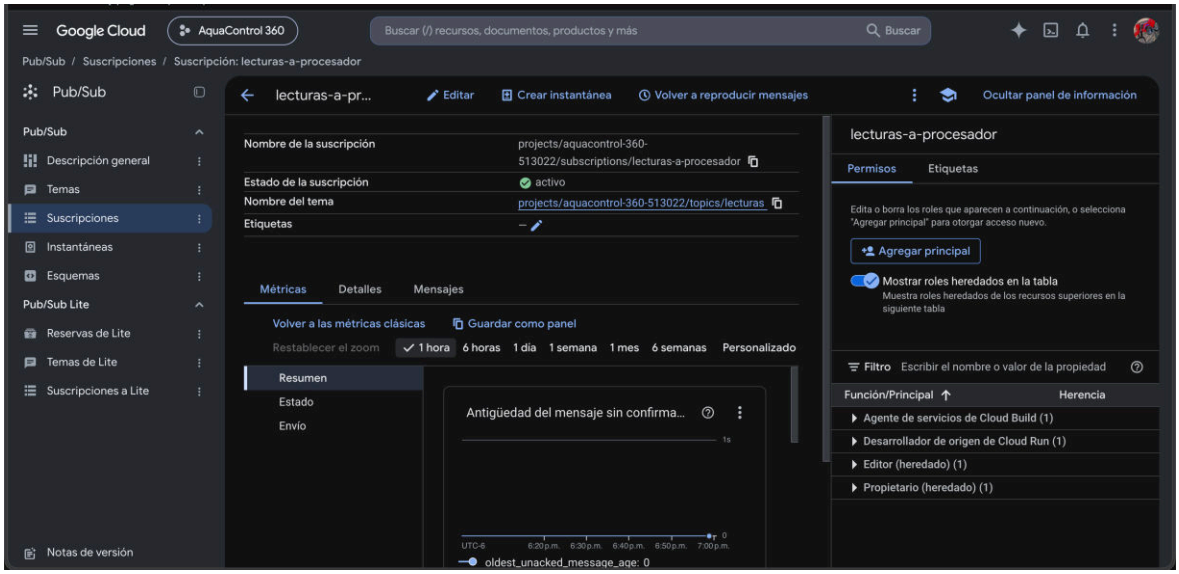


Figura 5. Pub/Sub: suscripción lecturas-a-procesador activa sobre el tópico lecturas, sin mensajes pendientes. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

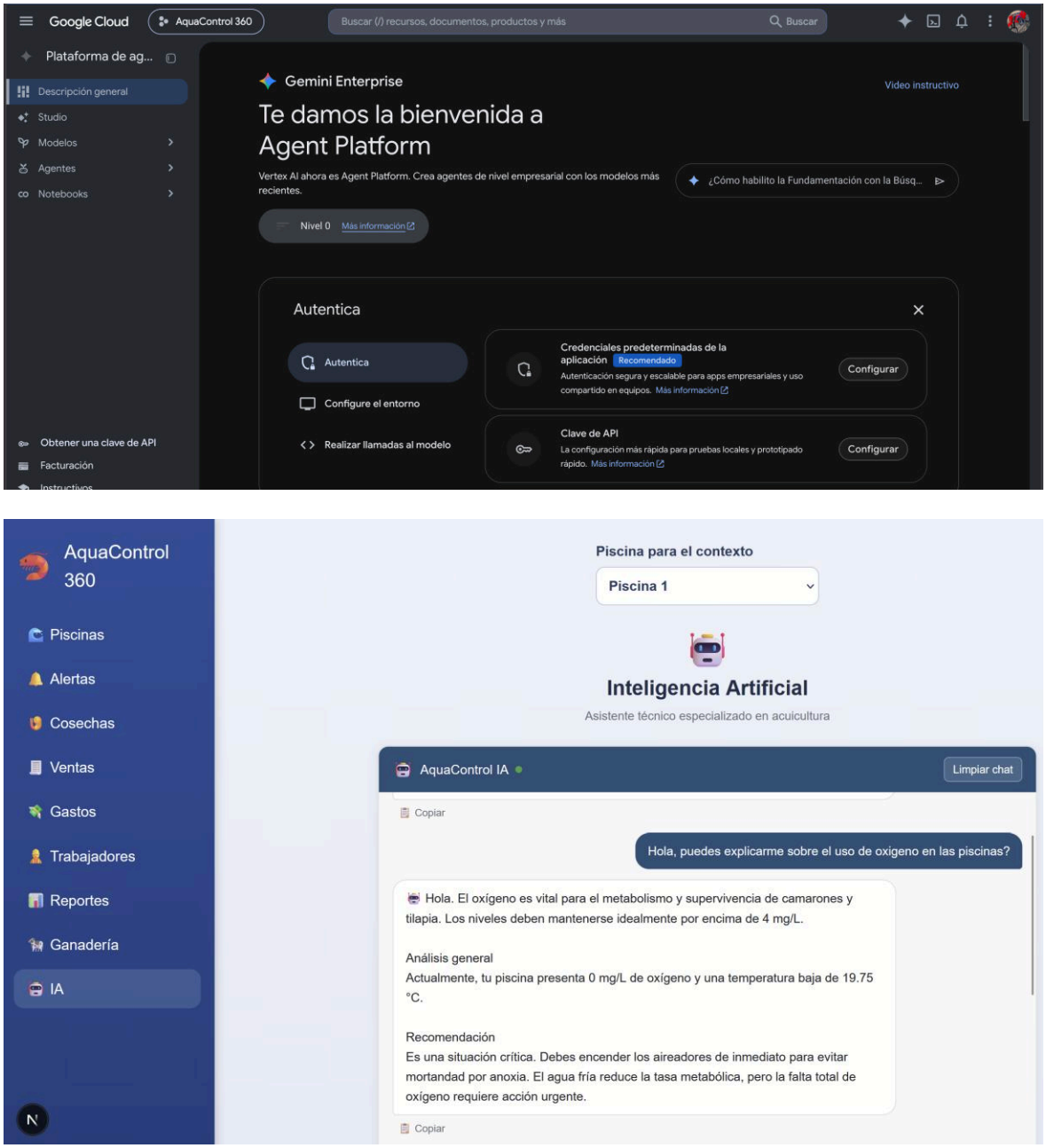


Figura 6. Vertex AI (Agent Platform) habilitado en el proyecto y asistente de la web respondiendo con Gemini a partir de las lecturas reales de la piscina 1. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

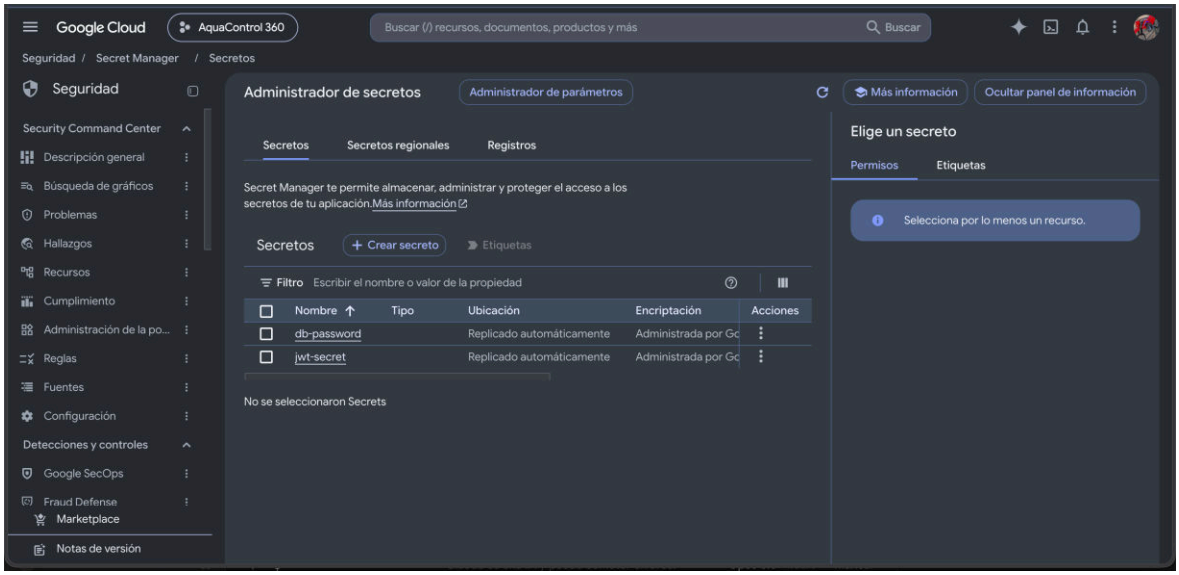


Figura 7. Secret Manager: secretos db-password y jwt-secret. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

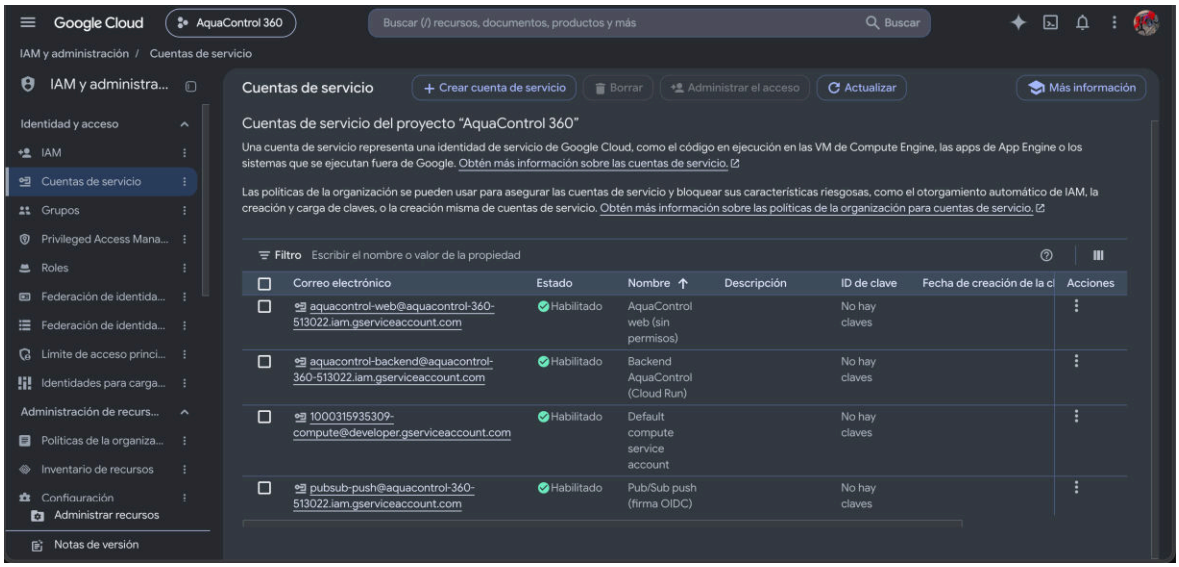


Figura 8. IAM: cuentas de servicio aquacontrol-backend, aquacontrol-web (sin permisos) y pubsub-push (firma OIDC), todas sin llaves descargadas. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

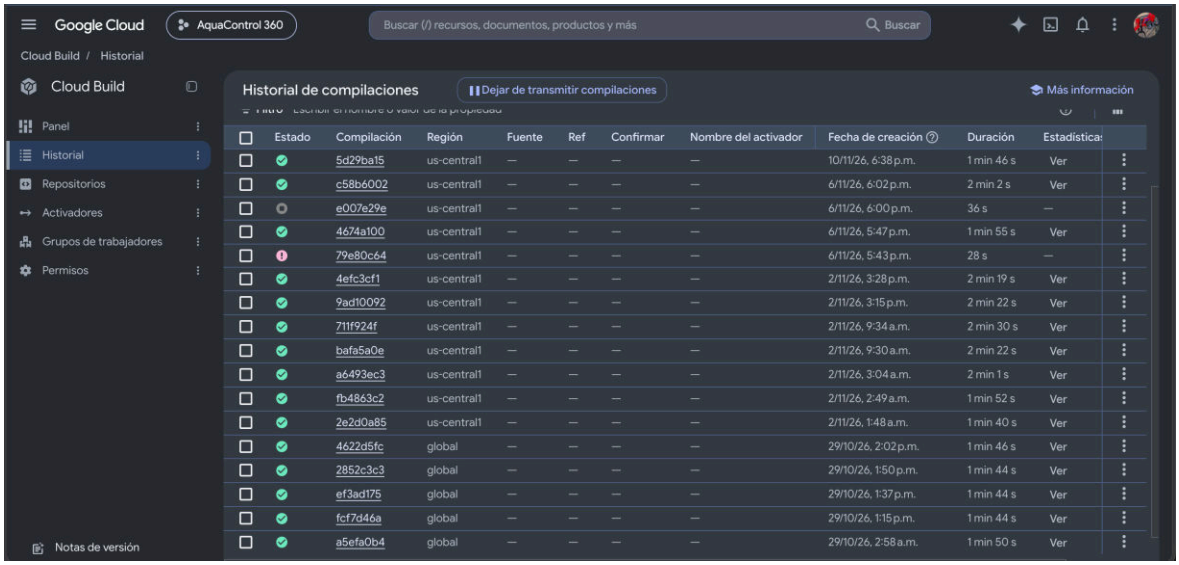


Figura 9. Cloud Build: historial de compilaciones en us-central1 con estado correcto. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

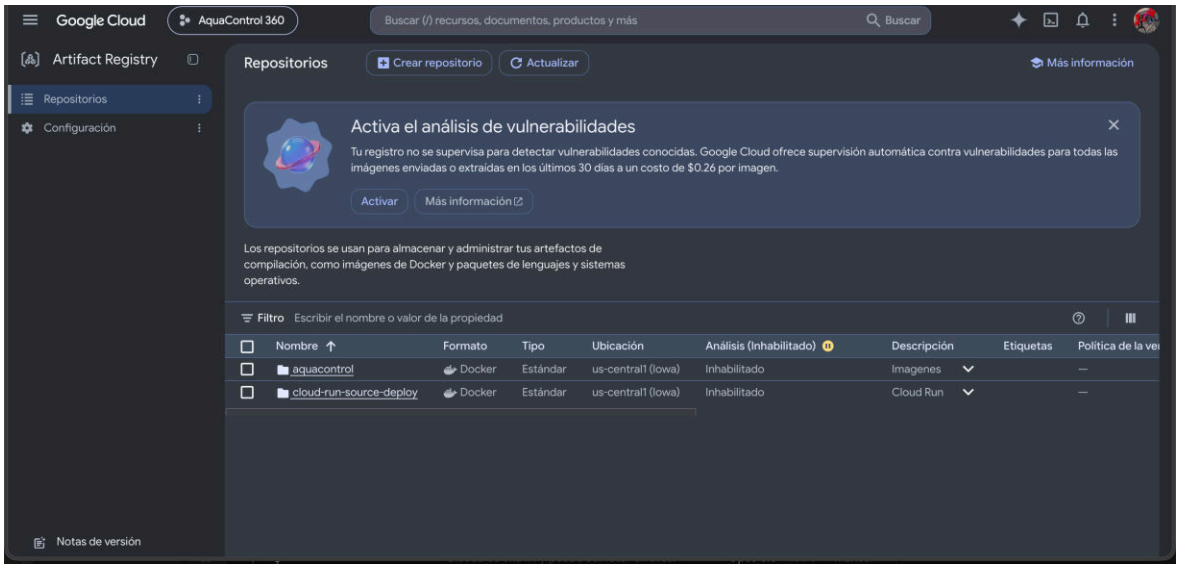


Figura 10. Artifact Registry: repositorios Docker aquacontrol y cloud-run-source-deploy. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

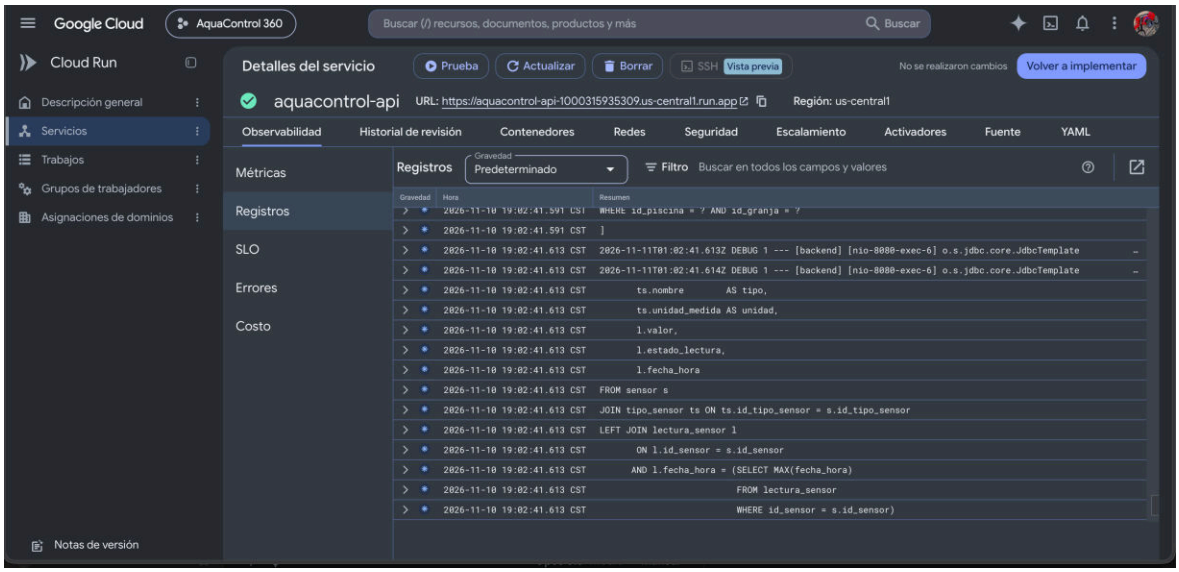


Figura 11. Cloud Logging: registros de Spring Boot del servicio aquacontrol-api. Captura de la consola de Google Cloud, proyecto aquacontrol-360-513022.

5. Flujos principales entre servicios

5.1 Lectura de un sensor y alerta de oxígeno

1. El ESP32-S3 lee el oxígeno disuelto y la temperatura. Si el oxígeno baja del umbral, enciende el aireador por su cuenta; esta decisión no depende de internet.

2. Cada minuto envía las lecturas a POST /api/iot/lecturas con su token de dispositivo.

3. La API valida el token, publica la lectura en el tópico lecturas de Pub/Sub y responde de inmediato al ESP32 con el estado de sus actuadores.

4. Pub/Sub entrega el mensaje por push a /api/pubsub/lecturas con un token OIDC firmado por Google en nombre de la cuenta de servicio pubsub-push. La API verifica la firma, la audiencia y la cuenta, y rechaza cualquier llamada que no cumpla.

5. La API guarda la lectura en Cloud SQL. Si el oxígeno es menor a 4 mg/L y no hay otra alerta pendiente del mismo sensor, crea una alerta.

6. La web y la app consultan las alertas nuevas y muestran la notificación al trabajador, que puede encender el aireador desde el teléfono.

7. El ESP32 consulta /api/iot/estado cada 3 segundos y aplica el comando, por lo que el aireador responde casi al instante.

5.2 Conteo de ganado con IA

1. La ESP32-CAM sube una foto cada 3 segundos a la API; la API guarda la última en Cloud Storage.

2. La web muestra esa foto como vista en vivo del potrero.

3. Al presionar Contar, la API copia la foto con su propio nombre en el bucket y se la envía a Gemini en Vertex AI.

4. Gemini devuelve la cantidad de animales y sus coordenadas; la API guarda el conteo en Cloud SQL junto con la ruta de la imagen.

5. La web dibuja un recuadro sobre cada animal y muestra el historial de conteos del potrero.

Este flujo muestra cuatro servicios trabajando juntos en una sola acción: Cloud Run, Cloud Storage, Vertex AI y Cloud SQL.

5.3 Despliegue de una nueva versión

El código vive en GitHub. Desde Cloud Shell se descarga la rama y se ejecuta el despliegue: Cloud Build compila el proyecto con su Dockerfile, guarda la imagen en Artifact Registry y Cloud Run crea una revisión nueva que recibe el 100 % del tráfico. Si la revisión nueva falla, se puede regresar a la anterior desde la consola de Cloud Run sin volver a compilar.

6. Decisiones de arquitectura

Decisión	Alternativas descartadas	Motivo
Cloud Run para la API y la web	Compute Engine, GKE, App Engine	Se paga por uso y escala a cero; no hay servidores que administrar ni parches de sistema operativo. GKE es demasiado complejo para dos servicios.
Cloud SQL con PostgreSQL	Firestore, Firebase	Los datos tienen muchas relaciones (granja, piscina, ciclo, muestreo, atarrayazo) y el curso exige SQL explícito y normalización. Una base relacional administrada da respaldos automáticos.
Pub/Sub entre el ESP32 y la base	Escribir directo en la base dentro de la	Desacopla la recepción del procesamiento: el ESP32 recibe su respuesta al instante y, si el guardado falla,

	solicitud	Pub/Sub reintenta.
Control local en el ESP32	Depender solo de la nube	Una caída de internet de noche no puede dejar sin aireación a la piscina.
Archivos en Cloud Storage	Guardar imágenes en PostgreSQL	Las imágenes harían crecer la base y su costo; en SQL solo se guarda la ruta.
Un solo backend para todos los clientes	Que cada cliente acceda a los datos por su cuenta	Las reglas de negocio y los permisos se escriben una sola vez y ningún cliente tiene credenciales de la base.

7. Instrucciones de despliegue paso a paso

Los comandos se ejecutan en Cloud Shell, con un usuario que tenga el rol de propietario del proyecto. Los valores entre signos de menor y mayor se reemplazan por los propios.

Paso 1. Proyecto y APIs

Se creó el proyecto aquacontrol-360-513022 con la cuenta de facturación del crédito de prueba y se activaron las APIs necesarias:

```
gcloud config set project aquacontrol-360-513022
gcloud services enable run.googleapis.com sqladmin.googleapis.com \
  secretmanager.googleapis.com artifactregistry.googleapis.com \
  cloudbuild.googleapis.com storage.googleapis.com iam.googleapis.com \
  pubsub.googleapis.com aiplatform.googleapis.com
```

Paso 2. Base de datos en Cloud SQL

```
gcloud sql instances create aquacontrol-db --database-version=POSTGRES_17 \
  --edition=ENTERPRISE --tier=db-f1-micro --region=us-central1 \
  --storage-type=SSD --storage-size=10GB --availability-type=zonal \
  --backup-start-time=08:00
gcloud sql databases create aquacontrol --instance=aquacontrol-db
gcloud sql users create aquacontrol_app --instance=aquacontrol-db --password=<CONTRASEÑA>
gcloud sql connect aquacontrol-db --user=postgres --database=aquacontrol
-- dentro de psql:
\i BD.sql
\i DatosPrueba.sql
```

Se eligió la edición Enterprise con db-f1-micro porque es la máquina más barata, y una sola zona porque la alta disponibilidad duplicaría el costo sin aportar al prototipo. El usuario aquacontrol_app es el que usa la API; el usuario postgres solo se usa para administrar.

Paso 3. Secretos en Secret Manager

```
printf "%s" "<CONTRASEÑA>" | gcloud secrets create db-password --data-file=-
openssl rand -base64 64 | tr -d '\n' | gcloud secrets create jwt-secret --data-file=-
```

Paso 4. Cuenta de servicio del backend con permisos mínimos

```
gcloud iam service-accounts create aquacontrol-backend \
  --display-name="Backend AquaControl (Cloud Run)"
SA=aquacontrol-backend@aquacontrol-360-513022.iam.gserviceaccount.com
gcloud projects add-iam-policy-binding aquacontrol-360-513022 \
  --member="serviceAccount:$SA" --role="roles/cloudsql.client"
gcloud secrets add-iam-policy-binding db-password \
  --member="serviceAccount:$SA" --role="roles/secretmanager.secretAccessor"
gcloud secrets add-iam-policy-binding jwt-secret \
  --member="serviceAccount:$SA" --role="roles/secretmanager.secretAccessor"
gcloud projects add-iam-policy-binding aquacontrol-360-513022 \
  --member="serviceAccount:$SA" --role="roles/aiplatform.user"
```

Paso 5. Bucket privado en Cloud Storage

```
gcloud storage buckets create gs://aquacontrol-360-513022-archivos \
  --location=us-central1 --uniform-bucket-level-access --public-access-prevention
gcloud storage buckets add-iam-policy-binding gs://aquacontrol-360-513022-archivos \
  --member="serviceAccount:$SA" --role="roles/storage.objectAdmin"
```

El permiso de administrador de objetos se otorga solo sobre este bucket, no sobre el proyecto.

Paso 6. Imagen del backend con Cloud Build y Artifact Registry

```
gcloud artifacts repositories create aquacontrol --repository-format=docker \
  --location=us-central1 --description="Imagenes del backend AquaControl"
git clone https://github.com/VegettoSsjBlue/ERP-para-Acuicultura.git aquacontrol
cd aquacontrol && git checkout BD-actualizada
gcloud builds submit backend \
  --tag us-central1-docker.pkg.dev/aquacontrol-360-513022/aquacontrol/backend:v1
```

Cada versión lleva una etiqueta nueva (v1 login, v2 comandos, v3 CORS, v4 muestreos), así se sabe qué versión corría en cada momento y se puede regresar a una anterior por su nombre.

Paso 7. Despliegue de la API en Cloud Run

```
gcloud run deploy aquacontrol-api \
  --image us-central1-docker.pkg.dev/aquacontrol-360-513022/aquacontrol/backend:v1 \
  --region us-central1 --service-account $SA \
  --set-env-vars "DB_URL=jdbc:postgresql:///aquacontrol?cloudSqlInstance=\
aquacontrol-360-513022:us-central1:aquacontrol-db&socketFactory=\
```

```
com.google.cloud.sql.postgres.SocketFactory,DB_USER=aquacontrol_app,\
GANADO_BUCKET=aquacontrol-360-513022-archivos" \
  --set-secrets "DB_PASSWORD=db-password:latest,JWT_SECRET=jwt-secret:latest" \
  --allow-unauthenticated --memory 512Mi --max-instances 2
```

Para las versiones siguientes se usa el despliegue desde el código fuente, que ejecuta Cloud Build y Artifact Registry automáticamente y conserva las variables y secretos ya configurados:

```
gcloud run deploy aquacontrol-api --source backend --region us-central1 --project
aquacontrol-360-513022
```

Prueba del despliegue:

```
curl -X POST https://aquacontrol-api-1000315935309.us-central1.run.app/api/auth/login \
  -H "Content-Type: application/json" -d '{"username":"<usuario>","password":"<clave>"}'
```

Si la respuesta trae el token, la API ya está leyendo Cloud SQL. El token sale firmado con HS512, la clave larga de Secret Manager, y no con la clave de desarrollo, lo que comprueba que el secreto de producción nunca estuvo en el código.

Paso 8. Pub/Sub para las lecturas con autenticación OIDC

```
gcloud pubsub topics create lecturas
gcloud pubsub topics add-iam-policy-binding lecturas \
  --member="serviceAccount:$SA" --role="roles/pubsub.publisher"


# Cuenta que firma los mensajes push (no tiene ningún otro permiso)
gcloud iam service-accounts create pubsub-push --display-name="Pub/Sub push (firma OIDC)"
PUSH_SA=pubsub-push@aquacontrol-360-513022.iam.gserviceaccount.com
URL_API="$(gcloud run services describe aquacontrol-api --region us-central1 \
  --format='value(status.url)')"
AUD="${URL_API}/api/pubsub/lecturas"


gcloud run services update aquacontrol-api --region us-central1 \
  --update-env-vars "PUBSUB_TOPIC_LECTURAS=lecturas,PUBSUB_PROJECT=aquacontrol-360-
513022,\
PUBSUB_PUSH_AUDIENCE=${AUD},PUBSUB_PUSH_SA=${PUSH_SA}"


gcloud pubsub subscriptions create lecturas-a-procesador --topic lecturas \
  --push-endpoint="${AUD}" --push-auth-service-account="${PUSH_SA}" \
  --push-auth-token-audience="${AUD}" \
  --ack-deadline=30 --min-retry-delay=10s --max-retry-delay=600s
```

Con esta configuración, Pub/Sub agrega a cada mensaje el encabezado Authorization con un token JWT firmado por Google. La API lo valida con TokenVerifier, de la librería de autenticación de Google: comprueba la firma, que el emisor sea Google, que la audiencia sea la URL del endpoint y que el correo del token sea el de pubsub-push. Así

no viaja ningún secreto en la URL, el token vence en una hora y no queda expuesto en los registros ni en la configuración de la suscripción.

Paso 9. Despliegue del sitio web

```
gcloud iam service-accounts create aquacontrol-web \
  --display-name="AquaControl web (sin permisos)"
cd my-app
gcloud run deploy aquacontrol-web --source . --region us-central1 \
  --service-account aquacontrol-web@aquacontrol-360-513022.iam.gserviceaccount.com \
  --set-env-vars API_URL=<URL_API> --no-allow-unauthenticated
gcloud run services add-iam-policy-binding aquacontrol-web --region us-central1 \
  --member=allUsers --role=roles/run.invoker
```

La web tiene su propia cuenta de servicio sin permisos, porque no necesita acceder a ningún recurso de Google Cloud: solo habla con la API. Si no se le asigna una, usaría la cuenta predeterminada de Compute Engine, que tiene rol de Editor sobre todo el proyecto. El servicio se despliega privado y se hace público en un paso aparte, que solo puede hacer el propietario.

Paso 10. Acceso del segundo integrante

```
gcloud projects add-iam-policy-binding aquacontrol-360-513022 \
  --member="user:andr[redacted] --role="roles/run.sourceDeveloper"
gcloud projects add-iam-policy-binding aquacontrol-360-513022 \
  --member="user:andr[redacted] --
role="roles/serviceusage.serviceUsageConsumer"
gcloud iam service-accounts add-iam-policy-binding \
  aquacontrol-web@aquacontrol-360-513022.iam.gserviceaccount.com \
  --member="user:andr[redacted] --role="roles/iam.serviceAccountUser"
```

Andrés puede desplegar la web, pero solo con la cuenta sin permisos, y no puede cambiar quién tiene acceso ni usar la cuenta del backend.

Paso 11. Clientes y nodos IoT

- En el firmware del ESP32-S3 y de la ESP32-CAM se configura la URL de la API y el token de cada dispositivo (registrado en la tabla dispositivo).
- En Android e iOS se configura la misma URL base de la API.
- La web usa la variable API_URL del servidor y guarda el token en una cookie httpOnly, por lo que el navegador nunca ve la URL interna ni el token.

Paso 12. Verificación y monitoreo

- Cloud Run, servicio aquacontrol-api, pestaña Registros: logs de Spring Boot con cada solicitud y error.
- Cloud Run, pestaña Métricas: solicitudes por segundo, latencia, errores 5xx e instancias activas.
- Cloud SQL, panel de la instancia: CPU, conexiones y almacenamiento.
- Pub/Sub, suscripción lecturas-a-procesador: mensajes sin confirmar, que deben quedar en cero cuando todo funciona.

Comandos para verificar los 10 servicios desde Cloud Shell:

```
gcloud run services list
gcloud sql instances list
gcloud storage buckets list --format="value(name)"
gcloud pubsub topics list; gcloud pubsub subscriptions list
gcloud services list --enabled | grep aiplatform
gcloud secrets list
gcloud iam service-accounts list
gcloud builds list --limit=5
gcloud artifacts repositories list
gcloud logging read 'resource.type="cloud_run_revision"' --limit=3
```

8. Consideraciones de seguridad

Medida	Cómo se aplica
Permisos mínimos (IAM)	Cuenta aquacontrol-backend solo con los roles que usa; el permiso de Storage es sobre un bucket y el de Pub/Sub sobre un tópico. La web corre con una cuenta sin permisos. El segundo integrante tiene roles limitados.
Secretos fuera del código	Contraseña de la base y clave de los JWT en Secret Manager. El repositorio público no contiene credenciales de producción y su historial se revisó antes de publicarlo.
Sin llaves JSON	Los servicios se autentican con su identidad de cuenta de servicio; no se descargó ninguna llave.
Conexión a la base	La API se conecta mediante el conector de Cloud SQL (socket factory) autenticado con IAM; la contraseña de la base solo existe en Secret Manager.
Autenticación de usuarios	Usuarios propios en PostgreSQL con contraseñas cifradas con BCrypt. Tokens JWT firmados con HS512 y con vencimiento.
Autorización por rol	Cada endpoint valida el rol (Dueño, Administrador, Trabajador) y que los datos pertenezcan a la granja del usuario.
Dispositivos IoT	Cada ESP32 se identifica con su propio token de dispositivo; un token no sirve para otra granja.
Endpoint de Pub/Sub	Autenticación OIDC: cada mensaje trae un JWT firmado por Google para la cuenta pubsub-push. La API verifica firma, emisor, audiencia y cuenta; sin un token válido responde 401. No hay secretos en la URL.
Credenciales de prueba	No se publican en documentos; se comparten por canal privado y se cambian después de cada entrega.
Archivos privados	Bucket con acceso uniforme y prevención de acceso público. Las imágenes se entregan a través de la API con sesión iniciada.

Web	Token en cookie httpOnly, puente /api/backend en el servidor de Next.js y CORS limitado a los orígenes del proyecto.
Consultas SQL	Consultas escritas a mano con parámetros (JdbcClient), lo que evita inyección SQL.
HTTPS	Cloud Run entrega HTTPS administrado en ambos servicios.
Disponibilidad	Respaldo diario automático de Cloud SQL y control local de aireación en el ESP32 ante fallas de red.

9. Estimación de costos

La estimación se hizo con los precios de lista de Google Cloud para us-central1, consultados en las páginas oficiales de precios, con los mismos parámetros que se usarían en la Google Cloud Pricing Calculator. Se calcularon tres escenarios y los totales se verificaron con un script de Python.

9.1 Precios unitarios utilizados

Servicio	Precio (USD)	Capa gratuita mensual
Cloud SQL db-f1-micro	0.0105 por hora (≈ 7.67 al mes)	No tiene
Cloud SQL disco SSD	0.17 por GiB al mes	No tiene
Cloud Run CPU	0.000024 por vCPU-segundo	180,000 vCPU-segundo
Cloud Run solicitudes	0.40 por millón	2 millones
Pub/Sub	40 por TiB	10 GiB
Cloud Storage Standard	0.02 por GiB al mes	5 GB
Vertex AI (Gemini)	0.30 entrada y 2.50 salida por millón de tokens*	No tiene
Secret Manager	0.06 por versión al mes	6 versiones
Cloud Build / Logging	Por uso	2,500 min / 50 GiB
Artifact Registry	0.10 por GB al mes	0.5 GB

* Se usó la tarifa de Gemini 2.5 Flash como cota alta. El sistema usa gemini-3.1-flash-lite, de la familia Flash-Lite, por lo que el costo real de IA debería ser igual o menor.

9.2 Supuestos de cada escenario

Escenario	Uso supuesto	Base de datos
Prototipo (actual)	1 granja, 1 ESP32 que envía cada 5 minutos, pocos usuarios, 300 preguntas a la IA al mes	db-f1-micro, 10 GB
10 granjas	10 ESP32 cada minuto, 50 usuarios, 3,000 preguntas a la IA	db-g1-small, 20 GB
100 granjas	100 ESP32 cada minuto, 500 usuarios, 30,000 preguntas a la IA	1 vCPU y 3.75 GB, 50 GB

Cada pregunta a la IA se estimó en 2,000 tokens de entrada y 500 de salida. En Cloud Run se supuso 0.15 s de CPU por solicitud con 1 vCPU y 512 MiB, sin contar la concurrencia, por lo que es una cota alta.

9.3 Costo mensual proyectado (USD)

Servicio	Prototipo	10 granjas	100 granjas
Cloud SQL	9.93	30.15	61.01
Vertex AI (Gemini)	0.56	5.55	55.50
Cloud Run	0.00	0.00	32.25
Cloud Storage	0.00	0.30	1.90
Artifact Registry	0.01	0.13	0.13
Pub/Sub, Secret Manager, Build, Logging	0.00	0.00	0.00
Total mensual	**10.50**	**36.13**	**150.79**
Costo por granja	**10.50**	**3.61**	**1.51**

9.4 Análisis de viabilidad

En el prototipo casi todo el costo es Cloud SQL, porque es el único servicio sin capa gratuita y cobra por hora aunque no se use; los demás servicios caben en las capas gratuitas. El crédito de prueba de 300 USD por 90 días cubre el proyecto con mucha holgura.

Al crecer, el costo por granja baja de 10.50 a 1.51 USD, porque la base es un costo fijo que se reparte entre más clientes. Con 100 granjas el costo que más crece es la IA, que llega a ser casi igual al de la base, y Cloud Run empieza a cobrar al superar los dos millones de solicitudes gratuitas. Pub/Sub sigue en cero incluso con más de cuatro millones de lecturas al mes. Con un cobro de suscripción de pocos dólares por granja, la arquitectura es financieramente viable.

Medidas para controlar el gasto: límite de 2 instancias en Cloud Run (lo que también protege a la base de demasiadas conexiones), escalado a cero cuando no hay tráfico y la máquina de base de datos más pequeña. Como medidas recomendadas para producción quedan una alerta de presupuesto en Cloud Billing, un tope diario de preguntas a la IA por usuario y una política de retención de lecturas.

Costo de una decisión: con cero instancias mínimas, Spring Boot tarda unos segundos en arrancar después de un tiempo sin uso. Mantener una instancia siempre encendida costaría cerca de 9.86 USD al mes; en el prototipo se aceptó esa espera para no pagarla.

10. Escalabilidad y manejo de recursos

- **Cómputo:** Cloud Run agrega instancias según la carga y vuelve a cero sin tráfico. Para crecer basta con subir el máximo de instancias.
- **Base de datos:** la instancia se puede cambiar a una máquina mayor con un comando y unos minutos de interrupción; a mayor escala se agregarían réplicas de lectura.
- **Lecturas:** Pub/Sub absorbe picos de mensajes, por lo que la API no se satura cuando muchos ESP32 envían al mismo tiempo.
- **Archivos:** Cloud Storage no tiene límite práctico de capacidad y solo cobra lo que se guarda.
- **Clientes:** agregar una nueva granja no requiere infraestructura nueva: es un registro más en la base, con sus usuarios, piscinas y dispositivos.

11. Limitaciones y trabajo futuro

Para ser transparentes con lo implementado, se listan los puntos que quedaron como mejora futura:

- Separar la identidad del backend en varias cuentas de servicio, una por componente (API, procesamiento de Pub/Sub, despliegue), para limitar aún más el impacto de una cuenta comprometida.
- Procesar Pub/Sub en un servicio de Cloud Run separado y privado, para que Cloud Run valide el token OIDC antes de llegar al código; hoy la misma API recibe el push y valida el token en el código.
- Activador automático de Cloud Build conectado a GitHub, para que cada push a la rama principal despliegue sin intervención manual.

- Reportes PDF mensuales generados con Cloud Run Jobs y Cloud Scheduler, guardados en Cloud Storage.
- Alerta de presupuesto, tope de preguntas a la IA y retención de lecturas detalladas por 90 días.
- Infraestructura como código con Terraform.
- Módulos de sifoneo, alimentación, mortalidad, inventario y compras, que están diseñados en la base de datos y aparecen en la interfaz como Próximamente.
- En el segundo nodo, el oxígeno disuelto y el pH se simulan por software porque solo se cuenta con un sensor de oxígeno.

12. Cumplimiento de la rúbrica

Criterio	Evidencia
Integración de servicios (50 %)	10 servicios de GCP integrados entre sí, incluidos cómputo (Cloud Run), almacenamiento (Cloud Storage) y base de datos (Cloud SQL), con capturas en la sección 4.1. Firebase no se usa.
Diseño de arquitectura (20 %)	Un solo backend, mensajería asíncrona con Pub/Sub, escalado a cero, archivos fuera de la base y control local en el borde (secciones 3, 6 y 10).
Documentación (10 %)	Diagrama (sección 3), servicios (4), despliegue paso a paso (7), seguridad (8) y costos (9).
Hitos (5 %)	M1 entregado el 11 de septiembre y M2 el 9 de octubre de 2026.
Presentación (10 %)	Presentación en clase el 2 de noviembre de 2026 y video de demostración (enlace en la sección 1).
Usabilidad y rendimiento (5 %)	Servicios en producción con HTTPS, autenticación y monitoreo; respuesta casi inmediata del aireador.

Referencias

Google Cloud. (s.f.-a). Cloud Run pricing. Recuperado el 21 de octubre de 2026, de https://cloud.google.com/run/pricing

Google Cloud. (s.f.-b). Cloud SQL pricing. Recuperado el 21 de octubre de 2026, de https://cloud.google.com/sql/pricing

Google Cloud. (s.f.-c). Cloud Storage pricing. Recuperado el 21 de octubre de 2026, de https://cloud.google.com/storage/pricing

Google Cloud. (s.f.-d). Free cloud features and trial offer. Recuperado el 21 de octubre de 2026, de https://docs.cloud.google.com/free/docs/free-cloud-features

Google Cloud. (s.f.-e). Implementa servicios desde el código fuente. Recuperado el 2 de noviembre de 2026, de https://docs.cloud.google.com/run/docs/deploying-source-code

Google Cloud. (s.f.-f). Pub/Sub pricing. Recuperado el 21 de octubre de 2026, de https://cloud.google.com/pubsub/pricing

Google Cloud. (s.f.-g). Secret Manager pricing. Recuperado el 21 de octubre de 2026, de https://cloud.google.com/secret-manager/pricing

Google Cloud. (s.f.-h). Vertex AI generative AI pricing. Recuperado el 21 de octubre de 2026, de https://cloud.google.com/vertex-ai/generative-ai/pricing

Google Cloud. (s.f.-i). Authentication for push subscriptions. Recuperado el 10 de noviembre de 2026, de https://cloud.google.com/pubsub/docs/authenticate-push-subscriptions